

Local Job Board Pipeline

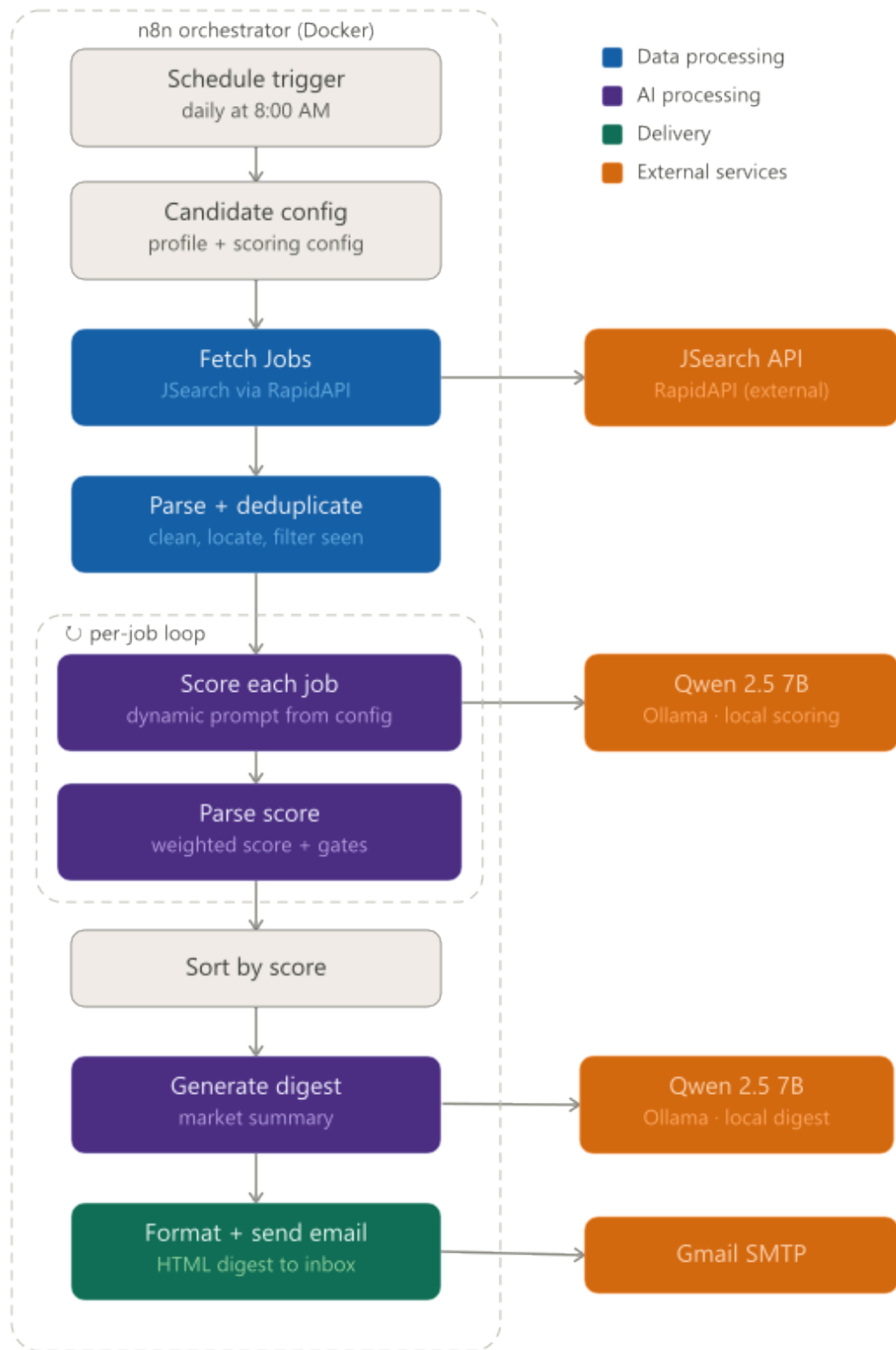
This project is a daily job-search pipeline that fetches, scores, and summarizes job postings automatically, finishing with a ranked digest to my inbox. It runs entirely on local infrastructure. The workflow is orchestrated by n8n, running inside Docker, and all LLM inferences are handled by local models via Ollama, so there are no per-token API costs and no data leaves the machine.

The pipeline is deliberately split between deterministic logic and LLM judgment. Location fit and seniority classification are computed from a centralized candidate profile before any model calls are made. It was found that the models performed poorly at these tasks, particularly location fit, and the deterministic approach is simple. The models are responsible for tasks they are well suited for: reading job descriptions, assessing skill match against a rubric, and writing concrete reasonings based on the postings actual requirements. Final scores are computed as a weighted average of five dimensions: skill match, seniority fit, location fit, growth signal, and compensation fit. There are hard gates in place for disqualifying roles that are independent of score.

The candidate profile, search parameters, scoring weights, city tier lists, seniority markers, and model names all live in a single `candidate_profile.json` config file. In this way, fine-tuning requires only a single change in one place.

Architecture

The pipeline runs on a daily schedule inside a self-hosted n8n instance. All LLM inference goes to Ollama on the host machine, reached from the container via `host.docker.internal`. The only outbound network calls are to the JSearch API, which offers 200 requests per month in their free tier, and Gmail's SMTP server for delivery.



Pipeline Stages

Candidate Config

This node runs as soon as the schedule is triggered. It is a Code node that returns the

candidate profile in structured JSON. It contains search parameters, target roles, skills lists, location tier maps, seniority markers, scoring weights, and model names. Every downstream node that needs any of these data can read it from this node by name rather than maintaining its own hardcoded data.

Fetch Jobs

A POST request is sent to the JSearch `/search-v2` endpoint via RapidAPI querying for recent postings in the target region. The specific query parameter values live inside the config file and can be altered there. Raw results come back as a paginated response object; the actual array of postings lives at `data.jobs`.

Parse Jobs

A Code node in which each raw job posting gets parsed in three major ways. First, the description text is sanitized of control characters, escaped sequences, and excess whitespace via regex. Secondly, the description gets truncated to 1200 words in order to keep prompt lengths manageable. Finally, two deterministic pre-computations are done that the LLM would otherwise handle unreliably. Location fit is computed by matching `job_city` against tiered lists from the config with preset scores mapped to each tiered list. Seniority is flagged by scanning the job title against a list of hard disqualifiers and amplifiers. In the current configuration, senior roles are filtered out and junior roles are signal boosted. A secondary scan of the description catches explicit experience thresholds (eg: 7+ years). Both values travel with each job item as `location_fit_hint` and `seniority_flag` so the LLM never has to derive them.

Deduplicate

Postings that have already been seen in a previous run get filtered out. Job ID's are stored in n8n's Static Data store (SQLite) with a Unix timestamp. This data persists across container restarts. A TTL pruning pass removes all entries older than thirty days so the store does not grow unboundedly. If all postings have been seen, the node emits a sentinel item that short-circuits the pipeline and triggers a minimal "no new postings" email.

Loop Over Items

Each job runs through three nodes in series:

Prepare Score Request

The full Ollama API request body is assembled. The system prompt is

constructed dynamically from the candidate profile rather than hardcoding it.

Score Job

A request is sent to Ollama's `/api/chat` endpoint with a JSON schema enforced by the `format` parameter, so the model is structurally constrained to return a valid response object.

Parse Score

The model output is received and immediately overwrites `location_fit` with the pre-computed value from the previous stage. A seniority gate is then applied if the `seniority_flag` was raised. The `seniority_fit` dimension is forced to the configured gate value (default is 1) and the final score is capped at the configured gate ceiling, regardless of how other dimensions scored. The final `match_score` is then computed as a weighted sum of five dimensions, with weights and gate values defined in `candidate_profile.json`. The LLM's raw holistic score is preserved as a separate `llm_score` field for comparison and evaluation purposes.

The five scoring dimensions and weights are as follows:

Dimension	Default weight	What it measures
<code>skill_match</code>	35%	Overlap between the posting's required tools and the candidate's demonstrated skills
<code>seniority_fit</code>	30%	Whether the role's seniority level matches the candidate's target career stage
<code>location_fit</code>	15%	Pre-computed tier score from config, never derived by the LLM
<code>growth_signal</code>	15%	Whether the role builds toward skills listed as growth targets in config
<code>comp_fit</code>	5%	Salary alignment where listed; defaults to neutral (5/10) when absent

Sort

All postings are sorted by `match_score` in descending order.

Prepare Digest Prompt

The second Ollama call is assembled consisting of a request for a brief market summary covering overall quality, trends across postings, and action items for the candidate. Crucially, it explicitly instructs the model not to restate individual job titles or scores, since a deterministic HTML table in the email already handles that more reliably.

Generate Digest

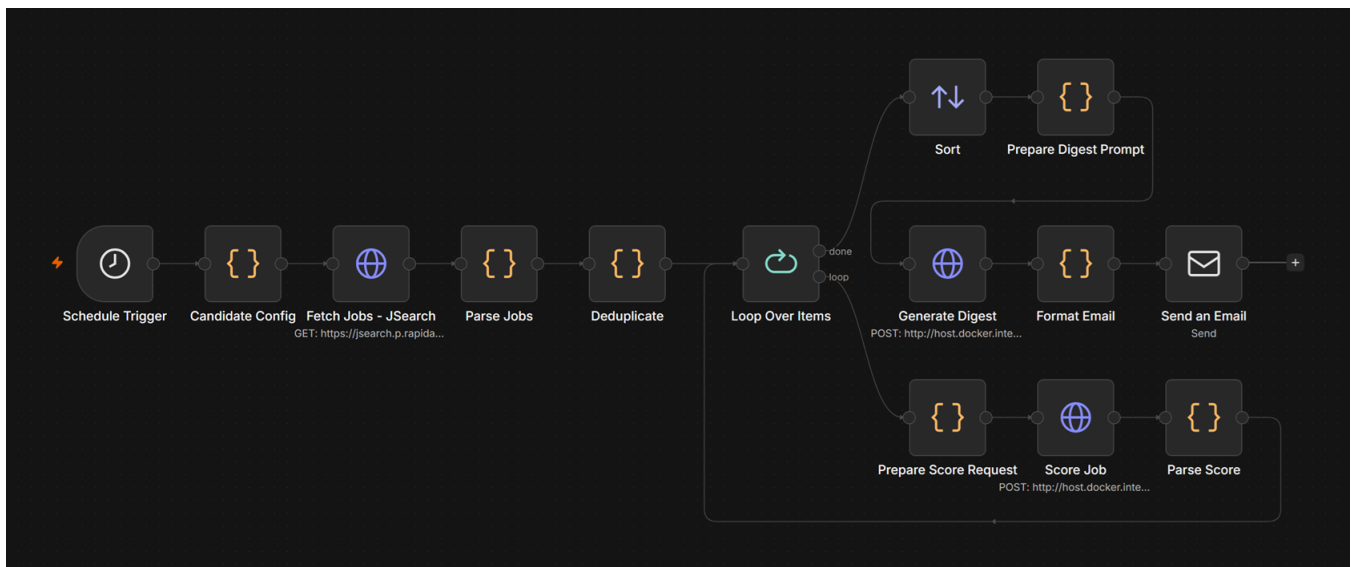
A POST request is sent to Ollama's `/api/chat` endpoint with the prompt prepared in the previous node. The temperature for this request is defined in the config and defaults to a slightly higher temperature than the Score Jobs request since natural variation in the prose is acceptable and desirable here.

Format Email

This node builds the HTML email. The top-ranked postings are selected by a score threshold with a fallback rule. All postings at or above the threshold score are shown when at least three are met, otherwise it shows the top three by rank. The score threshold is configurable (defaults to 7). Each entry in the table shows the title, company, location, weighted score, per-dimension breakdown, the LLM's reason, and the gate label when the seniority disqualifier is fired. The email footer prints the active weight breakdown from config making the scoring logic self-documenting in every digest.

Send an Email

Delivers the digest via Gmail SMTP.



Design Decisions

Several design decisions are shaped directly by the constraints of running a 7B model on consumer hardware. The model was found to be unreliable with geography lookups and consistent seniority flagging, so the architecture handles those explicitly with code. Ultimately, these are lookup problems, not comprehension, and treating them as the latter produced consistent failures: location would return zero consistently, and seniority assessments contained contradictions where a role would be flagged as disqualifying in `red_flags` while simultaneously being described as a good seniority fit in `reason`. In this case, the fix is not a better prompt, but reassigning the task.

Disqualifying roles is handled with hard gates rather than by blending their poor seniority score into the final weighted average. Seniority fit is more binary than the other dimensions. There may be wiggle room in the actual number of years of experience between what the candidate has and the role asks for, but a junior candidate is simply not qualified for a role that is explicitly labeled as Senior or Principal. Blending would produce a misleading middling result that buries the actual reason a role is not viable. The model's raw holistic score is still kept and displayed in the digest. This is useful for troubleshooting and testing, but a high `11m_score`, despite poor seniority fit, does describe a role that fits the more general candidate profile. A planned future addition is a secondary output that collects these high-score, gated postings as a labelled set, useful as a reference for understanding the market and as a foundation for evaluating and tuning the scoring model over time.

The system prompt in `Prepare Score Request` is assembled in code rather than stored in config, even though other tunable values live in `candidate_profile.json`. This is because the prompt interpolates from config at construction time. It reads the skill lists, gap lists, location base, and target roles to build itself dynamically. Moving the template to JSON would require a separate substitution layer to put those values back in. A config holds values that should be tunable without needing to understand the systems logic.

Configuration

All parameters live in `candidate_profile.json`, loaded at runtime by the `Candidate Config` node.

```

{
  "candidate": {
    "name": "...",
    "career_stage": "junior",
    "target_roles": ["Data Engineer", "ML Engineer", "..."],
    "strong_skills": ["Python", "SQL", "statistics", "..."],
    "building_skills": ["Power BI"],
    "skill_gaps": ["dbt", "Airflow", "Spark", "..."],
    "growth_targets": ["dbt", "Airflow", "cloud warehouses", "..."]
  },
  "location": {
    "base": "London, ON",
    "corridor": ["london", "kitchener", "waterloo", "..."],
    "gta": ["toronto", "oakville", "mississauga", "..."],
    "outer_gta": ["pickering", "ajax", "whitby", "..."],
    "longterm": "Vancouver",
    "scores": {
      "corridor": 9,
      "gta": 8,
      "outer_gta": 7,
      "remote_canada": 8,
      "longterm": 6,
      "canada_other": 4,
      "unknown": 5,
      "outside_canada": 1
    }
  },
  "seniority": {
    "hard_disqualifiers": ["senior", "sr.", "lead", "principal", "manager", "..."],
    "junior_signals": ["junior", "jr.", "entry", "new grad", "..."],
    "experience_threshold_years": 7
  },
  "scoring": {
    "weights": {
      "skill_match": 0.35,
      "seniority_fit": 0.30,
      "location_fit": 0.15,
      "growth_signal": 0.15,
      "comp_fit": 0.05
    }
  }
}

```



```

    },
    "gates": { "senior_role_max_score": 2, "senior_role_seniority_fit": 1 },
    "display_threshold": 7
  },
  "models": {
    "scoring": "qwen2.5:7b-instruct",
    "scoring_temperature": 0.15,
    "digest": "qwen2.5:7b-instruct",
    "digest_temperature": 0.3,
    "embedding": "nomic-embed-text"
  },
  "search": {
    "query": "junior data Ontario",
    "date_posted": "week",
    "num_pages": 1,
    "country": "ca",
    "dedup_ttl_days": 30
  }
}

```

`candidate` values are interpolated directly into the scoring system prompt — changes here propagate to every model call without touching node code. `location.scores` maps each tier name to a value between 0 and 10; `seniority.hard_disqualifiers` are matched against job titles before the model is called. `scoring.weights` must sum to 1.0. `models.embedding` is reserved for a planned retrieval stage and is not currently active.

Setup

Prerequisites

- [Docker Desktop](#) (includes Docker Compose)
- [Ollama](#) installed on the host machine
- A [RapidAPI](#) account with the [JSearch API](#) subscribed (free tier: 200 requests/month)
- A Gmail account with an [app password](#) configured for SMTP

1. Pull Ollama models

```
ollama pull qwen2.5:7b-instruct
ollama pull nomic-embed-text
```

These run on the host machine, not inside Docker. Ollama must be running before starting the workflow.

2. Start n8n

From the project root:

```
docker compose up -d
```

n8n will be available at `http://localhost:5678`. On first run it will prompt you to create an owner account.

3. Import the workflow

In the n8n editor: **Workflows** → **Add Workflow** → **Import from file**, and select `job-search-pipeline.json` from the repo.

4. Configure credentials

Two credentials are required. In n8n: **Settings** → **Credentials** → **Add Credential**.

JSearch API key

- Type: Header Auth
- Name: `JSearch RapidAPI`
- Header name: `X-RapidAPI-Key`
- Value: your RapidAPI key

In the **Fetch Jobs - JSearch** node, set Authentication to **Generic Credential Type** → **Header Auth** → **JSearch RapidAPI**.

Gmail SMTP

- Type: SMTP
- Host: `smtp.gmail.com` , Port: `465` , SSL: enabled
- User: your Gmail address
- Password: your app password (not your account password)

5. Configure the candidate profile

Edit `candidate_profile.json` to reflect your own profile, target roles, location tiers, and scoring preferences. The `search.query` field controls what JSearch fetches each day.

6. Activate the workflow

Open the imported workflow in the n8n editor and toggle it to **Active**. It will run daily at the time configured in the Schedule Trigger node. To test immediately, open the workflow and click **Execute Workflow**.